

Jenkins

The Complete Guide to CI/CD — Beginner to Proficient

Installation, jobs, pipelines, triggers, and automation — explained with a running scenario, diagrams, and simple → moderate → advanced coverage.

A practical, hands-on reference

How to Read This Guide

This guide takes you from never having used Jenkins to being genuinely proficient with continuous integration and delivery. Work through it in order the first time, then use the cheat sheet at the end as a day-to-day reference.

Each important feature is explained in three parts: **what it does** (the idea), **how to use it** (the exact steps or configuration), and **when to use it** (the situation that calls for it). Topics are grouped by level:

Simple — install Jenkins, finish the setup wizard, and run your first job.

Moderate — pipelines as code, the Jenkinsfile, and automatic build triggers.

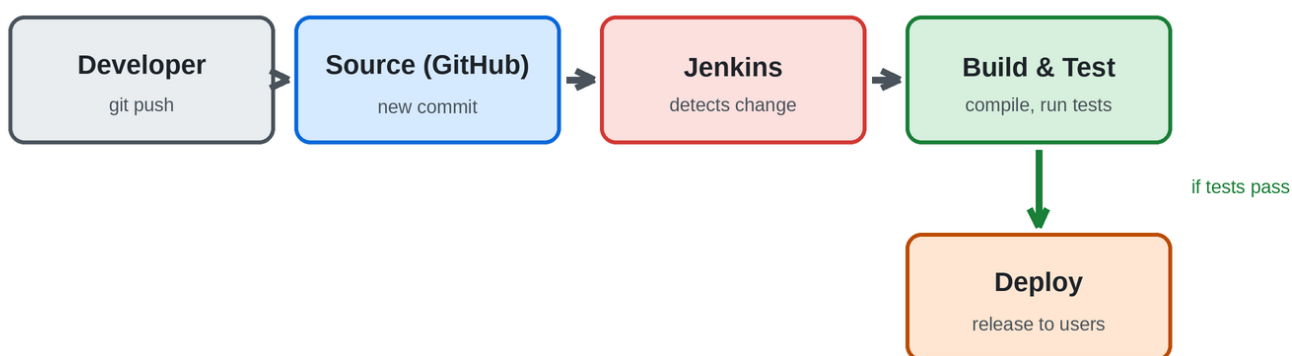
Advanced — agents, credentials, shared libraries, parallel stages, and keeping Jenkins healthy and secure.

A running **Scenario** follows a fresher named Aisha setting up CI/CD for a small web app called “HelloApp”. Yellow **Note** boxes flag the mistakes beginners actually make.

1. What is Jenkins & Why CI/CD?

Imagine a team of three developers working on the same application. Every Friday, one person collects everyone's code, pastes it together, compiles it, runs the tests by hand, and copies the result onto a server. If something breaks, nobody is sure whose change caused it, and the afternoon disappears into guesswork. That ritual is slow, stressful, and easy to get wrong. CI/CD exists to make that pain disappear by letting a machine do the repetitive work the same way, every time.

The CI/CD Pipeline: What Jenkins Automates



Jenkins runs the same steps automatically on every change — no manual builds, fast feedback.

Figure 1 — On every change, Jenkins fetches the code, builds and tests it, and (if the tests pass) deploys it — automatically.

Continuous Integration (CI)

Continuous Integration means every developer's changes are merged into a shared place frequently — often several times a day — and each time, the code is automatically built and tested. Because integration happens in small, frequent steps, problems surface within minutes of the change that caused them, while the author still remembers what they did.

Continuous Delivery / Deployment (CD)

Continuous Delivery takes the tested code and automatically prepares it so it is always ready to release. Continuous Deployment goes one step further and pushes every change that passes the tests straight to users, with no manual button-press. Together, CI and CD turn “release day” from a nerve-wracking event into a routine, repeatable action.

So what is Jenkins?

Jenkins is a free, open-source **automation server**. Think of it as a tireless assistant that watches your code and, whenever something changes, runs the exact sequence of steps you taught it: fetch the latest code, build it, run the tests, and (if you want) deploy it. It runs on Windows, macOS, and Linux, and is extended by a huge library of **plugins** — add-ons that teach it to talk to Git, Docker, cloud providers, Slack, and thousands of other tools.

Scenario: Throughout this guide we help a fresher named Aisha set up Jenkins for a tiny web app called “HelloApp” that lives on GitHub. By the end, Jenkins automatically builds, tests, and deploys HelloApp every time she pushes code.

2. Installing Jenkins

Jenkins usually runs as a standalone application in its own process. It ships as a single Java program bundled with a lightweight web server, so once Java is present you can start it on almost any operating system. Pick the option that matches your machine.

Prerequisite: Java (JDK)

What: Jenkins runs on Java, so a supported JDK must be installed first. **How:** recent Jenkins needs Java 17 or 21; Eclipse Temurin is a good free choice. **When:** always, before installing Jenkins.

```
java -version

# healthy output looks like:
openjdk version "21.0.3" 2024-04-16
```

A version older than 17 (or no Java at all) is the single most common reason Jenkins refuses to start. Install a supported JDK first.

Windows

1. Download the Jenkins Windows installer (.msi) from jenkins.io.
2. Run it — it installs Jenkins as a Windows service that starts with the computer.
3. Open `http://localhost:8080` in a browser.

macOS (Homebrew)

```
brew install jenkins-lts
brew services start jenkins-lts
```

Jenkins is then at `http://localhost:8080`. Stop it later with `brew services stop jenkins-lts`.

Linux (Debian / Ubuntu)

```
# add the Jenkins repo, then install
sudo apt-get update
sudo apt-get install -y jenkins

sudo systemctl start jenkins
sudo systemctl status jenkins
```

Running the WAR file directly

The most portable option — no installer, no service, just Java running one file. Great for trying Jenkins quickly:

```
java -jar jenkins.war

# run on a different port
java -jar jenkins.war --httpPort=9090
```

Leave that terminal open — closing it stops Jenkins.

Docker option

```
docker run --name jenkins --rm --detach \  
  --publish 8080:8080 --publish 50000:50000 \  
  --volume jenkins_home:/var/jenkins_home \  
  jenkins/jenkins:lts-jdk21
```

Port **8080** is the web interface; **50000** is used by build agents. The **jenkins_home** volume keeps your configuration safe even if the container is removed.

3. First Launch & Setup Wizard

However you installed it, the first time you open <http://localhost:8080> Jenkins runs a short one-time setup wizard.

Unlock Jenkins

What: Jenkins generates a random admin password on first startup and asks you to paste it, to prove you own the machine. **Where to find it:**

```
# Linux / macOS  
cat /var/lib/jenkins/secrets/initialAdminPassword  
  
# Windows (default service install)  
type C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

If you started from the WAR file or Docker, the same password is printed in the startup logs.

Install suggested plugins

Choose **Install suggested plugins** — it installs a sensible default set (Git, Pipeline, and more) that covers almost everything in this guide. You can add or remove plugins later.

Create admin user

Create your own administrator account (username, password, name, email) rather than keeping the random password. The wizard then confirms the Jenkins URL, which stays <http://localhost:8080> for local use.

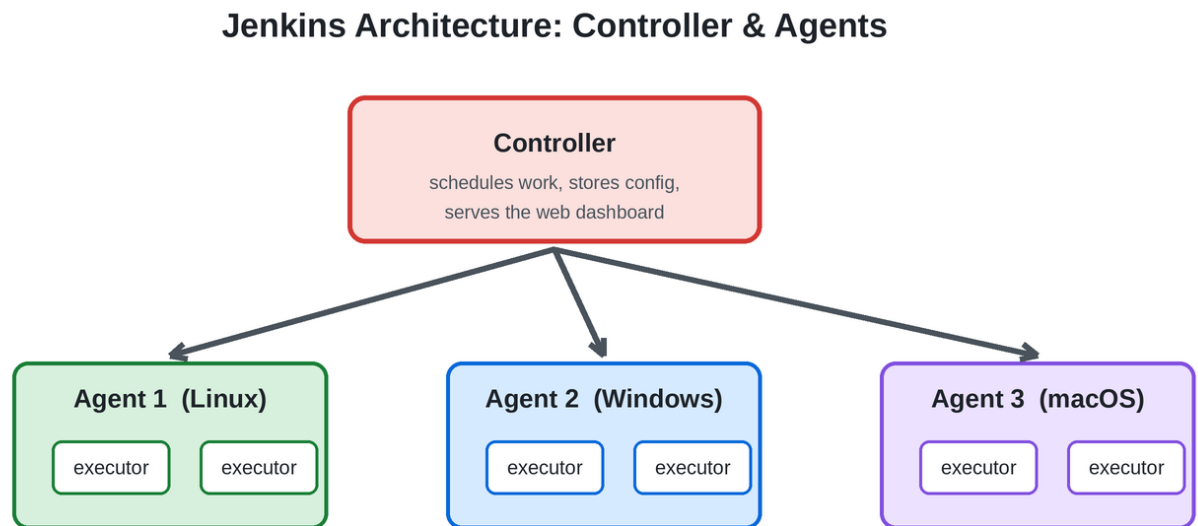
Dashboard tour

- **New Item** — create a new job or pipeline (top-left).
- **Jobs list** — the centre of the screen; each job shows its last build status (blue/green = success, red = failed).
- **Build Queue & Build Executor Status** — lower-left panels showing what is waiting and what is running.
- **Manage Jenkins** — settings for plugins, users, credentials, and system configuration.

Scenario: Aisha installs Jenkins with Homebrew, opens `http://localhost:8080`, pastes the initial admin password, picks “Install suggested plugins”, and creates an admin user “aisha”. She now sees an empty dashboard, ready for her first job.

4. Key Jenkins Concepts

A handful of terms appear everywhere in Jenkins. The diagram shows how the main pieces fit together.



The controller hands builds to agents. Each executor runs one build at a time, so many builds run in parallel.

Figure 2 — The controller schedules and stores everything; agents run the actual builds; each executor on an agent runs one build at a time.

Term	What it means
Controller	The main Jenkins server — stores configuration, schedules work, and serves the dashboard. (Older docs call it the “master”.)
Node / Agent	A machine that runs the work. The controller can run work itself or hand it to agent machines so builds run in parallel.
Executor	A slot on a node for running one build at a time. Two executors = two builds at once.
Job / Project	A unit of work you configure — e.g. “build and test HelloApp”. Freestyle and Pipeline are both kinds of jobs.
Build	One run of a job, numbered #1, #2, #3... so you can review the history of each run.
Workspace	A folder on the node where Jenkins checks out code and runs the build. Each job gets its own.

Term	What it means
Plugin	An add-on that extends Jenkins — e.g. the Git plugin to fetch code, or Docker to build images.
SCM	Source Code Management — the system holding your code (Git/GitHub). Jenkins pulls from it before building.

5. Your First Job (Freestyle)

A **Freestyle project** is the simplest kind of job — you configure it by filling in forms and ticking boxes, no code required. Perfect for your very first build.

Create the project

- On the dashboard, click New Item.
- Enter a name, e.g. HelloApp-build.
- Choose Freestyle project and click OK.

Add a source-code (Git) step

In **Source Code Management** select **Git** and paste your repo URL. Choose the branch to build, e.g. */main.

```
https://github.com/aisha/helloapp.git
```

Add a build step

Under **Build Steps** click **Add build step** and choose “Execute shell” (Linux/macOS) or “Execute Windows batch command”. Enter the commands that build and test your app:

```
npm install
npm test
npm run build
```

Click Save — the job now knows where the code is and what to do with it.

Run it & read the console output

Click **Build Now**. A new entry appears in **Build History** (build #1). Click the build number, then **Console Output** to watch, line by line, exactly what Jenkins did.

- A blue/green ball = success.
- A red ball = failed — the console output shows which command failed and why.

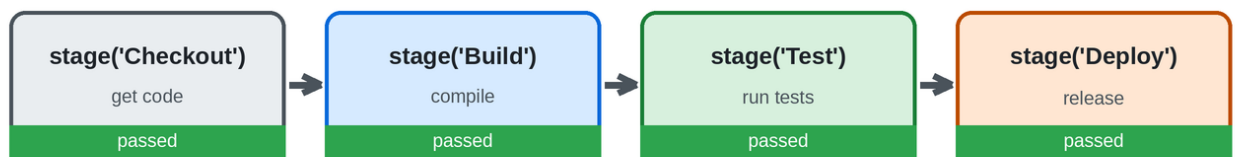
The Console Output is your best friend. When a build fails, read it from the bottom up — the last error is almost always the real cause.

Scenario: Aisha creates HelloApp-build, points it at her GitHub repo, adds an “npm install && npm test” step, and clicks Build Now. Build #1 fails on a missing dependency. She fixes package.json, pushes, builds again, and build #2 goes green — her first automated build works.

6. Pipelines

Freestyle jobs are fine for simple tasks, but real projects have several stages — build, test, deploy — and you want to see how each went. That is what a Pipeline gives you.

A Declarative Pipeline: Stage View



Each stage is one phase. Jenkins shows a box per stage, so you see exactly where a run succeeded or failed.

Figure 3 — A pipeline is a series of named stages. Jenkins draws one box per stage so you instantly see where a run passed or failed.

Pipeline vs Freestyle

A **Pipeline** describes your whole build process as code, broken into clear **stages**. Jenkins shows each stage as a box in a Stage View, so you can see “Build passed, Test passed, Deploy failed” at a glance. Being code, it can be stored with your app and reviewed like any change.

The Jenkinsfile

The pipeline definition lives in a file named **Jenkinsfile** in the root of your repository. Keeping it in the repo (“Pipeline as Code”) means the build process is versioned with the code it builds — check out an old version of the app and you get the matching build process too.

Declarative vs Scripted

Declarative — a structured, easy-to-read format with a fixed skeleton (pipeline { ... }). Recommended for almost everyone, especially beginners.

Scripted — full Groovy programming with loops and conditionals. More powerful but harder to read; reach for it only when Declarative cannot express what you need.

A worked Declarative pipeline

A complete beginner-friendly **Jenkinsfile** for HelloApp with three stages:

```
pipeline {
    agent any                // run on any available node
```

```

stages {
    stage('Build') {
        steps {
            echo 'Installing and building...'
            sh 'npm install'
            sh 'npm run build'
        }
    }
    stage('Test') {
        steps { sh 'npm test' }
    }
    stage('Deploy') {
        steps { sh './deploy.sh' }
    }
}

post {
    success { echo 'Pipeline finished successfully!' }
    failure { echo 'Something failed – check the stage view.' }
}
}

```

Top to bottom: the agent line says where to run; each stage is a named phase shown as its own box; the steps are the actual commands; and the post block runs cleanup or notifications depending on whether the whole pipeline passed or failed.

Scenario: Aisha moves HelloApp from a Freestyle job to a Pipeline, committing the Jenkinsfile above to her repo. Now a run shows three boxes — Build, Test, Deploy. When Deploy once goes red, she knows instantly that build and tests were fine and only deployment broke.

7. Automating Builds (Triggers)

So far Aisha clicks “Build Now” herself. The real power of Jenkins is triggers — rules that start a build automatically.

Poll SCM

What: Jenkins checks the repo on a schedule and builds only if something changed. **When:** a simple option when webhooks are not available.

```
H/5 * * * * # check every 5 minutes
```

The **H** (hash) spreads load so jobs do not all poll at the same second. Polling is simple but slightly wasteful — Jenkins keeps asking even when nothing changed.

GitHub webhooks

What: instead of Jenkins asking, GitHub tells Jenkins the instant you push. **How:** add a webhook in the GitHub repo settings pointing at your Jenkins URL, and enable “GitHub hook trigger” on the job. **When:** the preferred approach — builds start within seconds, no wasted polling.

Webhooks need your Jenkins server to be reachable from GitHub. On a laptop behind a home router that usually means a tunnelling tool or cloud-hosted Jenkins; on a company server it is normally already reachable.

Scheduled builds (cron)

What: build at a fixed time regardless of changes (e.g. a nightly build). The five fields are **minute** **hour** **day-of-month** **month** **day-of-week**.

Schedule	Meaning
H 2 * * *	Once a day, around 2 AM.
H 2 * * 1-5	Around 2 AM, Monday to Friday only.
H/30 * * * *	Roughly every 30 minutes.

Parameterized builds

What: the build asks for input first — like which environment to deploy to. In a pipeline, declare a **parameters** block:

```
pipeline {
  agent any
  parameters {
    choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Where
to deploy')
  }
  stages {
    stage('Deploy') { steps { echo "Deploying to ${params.ENV}" } }
  }
}
```

Now building shows a dropdown to pick dev, staging, or prod, and the pipeline uses your choice.

Scenario: Aisha adds a GitHub webhook so HelloApp builds within seconds of every push, and a nightly cron build at 2 AM that runs the full test suite. She never clicks “Build Now” again.

8. Controlling the Build

A few building blocks make pipelines genuinely useful day to day.

Environment variables

What: values available to every step. Define your own in an **environment** block; Jenkins also provides built-ins like **BUILD_NUMBER** and **BRANCH_NAME**.

```
pipeline {
  agent any
  environment { APP = 'HelloApp'; REGION = 'us-east-1' }
  stages {
    stage('Info') { steps { echo "Building ${APP} #${BUILD_NUMBER}" } }
  }
}
```

Archiving artifacts & test results

What: save build outputs (a .jar, a zip, a report) and publish test results so Jenkins can show pass/fail trends. **When:** almost always — it is how you keep and find what a build produced.

```

post {
    always {
        junit '**/test-results/*.xml' // publish test results
        archiveArtifacts artifacts: 'dist/**' // keep build output
    }
}

```

post conditions

The **post** block runs after the stages, with conditions: **always**, **success**, **failure**, **unstable**, and **changed**. Use it for notifications and cleanup — e.g. message the team on failure.

```

post {
    failure { echo 'Build failed – notify the team here (Slack/email plugin).' }
    success { echo 'All green!' }
}

```

when — conditional stages

Run a stage only in certain cases — e.g. deploy only on the main branch:

```

stage('Deploy') {
    when { branch 'main' }
    steps { sh './deploy.sh' }
}

```

Scenario: Aisha's pipeline now archives the built HelloApp bundle, publishes its test report so she can see trends, deploys only from main, and messages the team on failure — all declared in the Jenkinsfile.

9. Building CI/CD Pipelines End-to-End

This section ties everything together with complete, working examples — first a simple project (shown as both a Freestyle job and a pipeline so you can compare), then a full complex project with every stage a real team uses. Treat these as templates you can copy and adapt.

What makes a good pipeline

Whatever the project size, a solid CI/CD pipeline follows the same shape:

- Fast feedback first — put quick checks (compile, unit tests) early so failures surface in seconds, not minutes.
- Fail fast — if an early stage fails, later stages should not run.
- Build once, promote the same artifact — build the app a single time, then deploy that exact artifact to each environment (never rebuild per environment).
- Separate CI from CD — continuous integration (build + test) runs on every change; deployment is gated (branch rules, approvals).
- Make it repeatable — everything in the Jenkinsfile, versioned with the code; no manual steps that only live in someone's head.
- Clean up and report — always publish test results, archive artifacts, and notify on failure.

Rule of thumb: a stage should answer one clear question (“does it compile?”, “do tests pass?”, “is it deployed?”). If a stage does five unrelated things, split it so the Stage View pinpoints failures.

Example A — a simple project

The project: a small static website or Node.js app in one GitHub repo. Goal: on every push, install dependencies, run tests, and build. This is pure CI — no deployment yet. We show it two ways.

A1 — as a Freestyle job (no code)

Good for a first job or a very simple task. Configure it entirely through the UI:

7. New Item → name “website-ci” → Freestyle project.
8. Source Code Management → Git → paste the repo URL, branch */main.
9. Build Triggers → tick “GitHub hook trigger for GITScm polling” (build on push).
10. Build Steps → Execute shell → enter the commands below.
11. Post-build Actions → “Publish JUnit test result report” → test-results/*.xml.

```
npm install
npm test
npm run build
```

Save, then Build Now once to confirm it works. From then on, every push triggers it automatically. This is the quickest way to get a project under CI.

A2 — the same project as a pipeline (recommended)

The pipeline version does the same thing but lives in a **Jenkinsfile** in the repo, so it is versioned, reviewable, and shows a Stage View. Create a Pipeline job pointing at the repo, or paste this as the pipeline script:

```
pipeline {
  agent any

  triggers { githubPush() }           // build when GitHub notifies a push

  stages {
    stage('Checkout') {
      steps { checkout scm }          // get the code
    }
    stage('Install') {
      steps { sh 'npm ci' }           // clean, reproducible install
    }
    stage('Test') {
      steps { sh 'npm test' }
    }
    stage('Build') {
      steps { sh 'npm run build' }
    }
  }

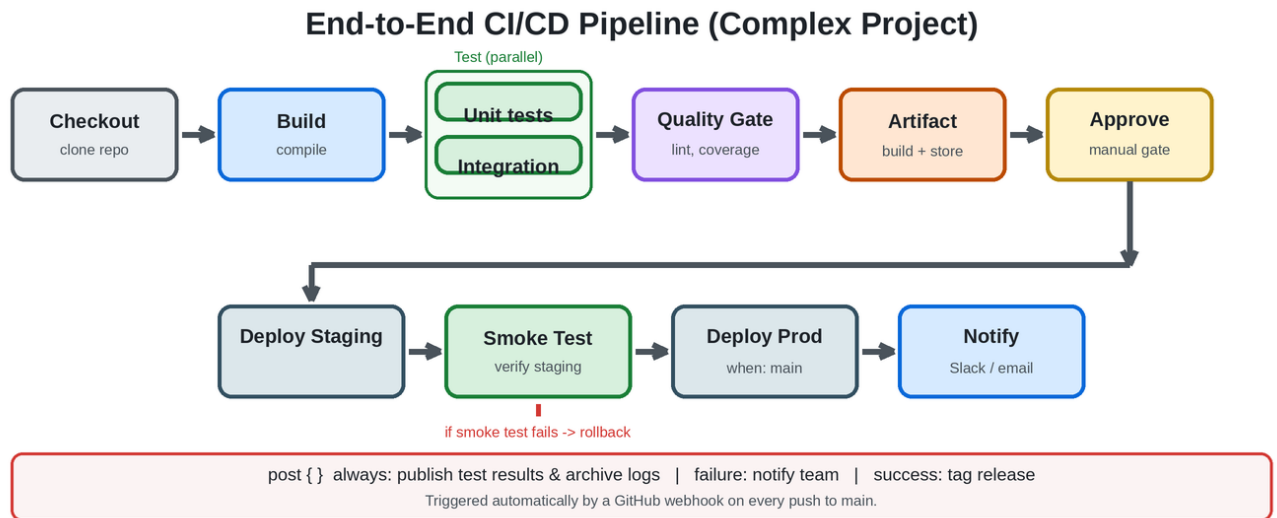
  post {
    always { junit 'test-results/*.xml' } // publish results
    success { echo 'CI passed ✓' }
    failure { echo 'CI failed – check the stage view' }
  }
}
```

Why the pipeline is better even for a simple project: the **Jenkinsfile** is reviewed in pull requests like any code, it travels with the repo (clone it anywhere and CI just works), and the Stage View shows exactly which step failed.

Scenario: Aisha starts HelloApp as a Freestyle “website-ci” job to get going in five minutes. A week later she moves the exact same three steps into a Jenkinsfile (version A2), commits it, and deletes the Freestyle job — now her CI is code, reviewed alongside the app.

Example B — a complex, multi-service project

The project: a real application with a build step, two kinds of tests, code-quality checks, a versioned artifact, deployment to staging then production, a manual approval before production, notifications, and automatic rollback if the release misbehaves. The diagram shows the whole flow.



Each box is a pipeline stage. Tests run in parallel; a manual approval gates production; failures roll back and notify.

Figure 4 — A production-grade pipeline: build once, test in parallel, gate quality, store the artifact, require manual approval, deploy staging → prod, roll back on failure, and always report.

The end-to-end Jenkinsfile

This is a complete Declarative pipeline implementing the diagram. Read the comments — each maps to a principle above.

```

pipeline {
    agent none // each stage picks its own agent

    options {
        timestamps() // timestamp every log line
        buildDiscarder(logRotator(numToKeepStr: '20')) // keep last 20 builds
        timeout(time: 30, unit: 'MINUTES') // fail if it hangs
    }

    environment {
        APP = 'shop-api'
        REGISTRY = 'registry.example.com'
    }

    triggers { githubPush() } // CI: run on every push

    stages {
        stage('Checkout') {
            agent { label 'linux' }
            steps { checkout scm }
        }

        stage('Build') { // build ONCE, reuse everywhere
            agent { label 'linux' }
            steps {
                sh 'make build'
                sh "docker build -t $REGISTRY/$APP:${GIT_COMMIT} ."
            }
        }
    }
  
```



```

stage('Test') {                                     // run the two test suites in parallel
    parallel {
        stage('Unit') {
            agent { label 'linux' }
            steps { sh 'make test-unit' }
            post { always { junit 'reports/unit/*.xml' } }
        }
        stage('Integration') {
            agent { label 'linux' }
            steps { sh 'make test-integration' }
            post { always { junit 'reports/integration/*.xml' } }
        }
    }
}

stage('Quality Gate') {                             // lint + coverage threshold
    agent { label 'linux' }
    steps { sh 'make lint'; sh 'make coverage-check' }
}

stage('Publish Artifact') { // push the image built above
    agent { label 'linux' }
    steps {
        withCredentials([usernamePassword(credentialsId: 'registry-login',
            usernameVariable: 'U', passwordVariable: 'P')]) {
            sh 'echo "$P" | docker login $REGISTRY -u "$U" --password-stdin'
            sh "docker push $REGISTRY/$APP:${GIT_COMMIT}"
        }
    }
}

stage('Deploy Staging') {
    agent { label 'linux' }
    steps { sh "./deploy.sh staging ${GIT_COMMIT}" }
}

stage('Smoke Test') {
    agent { label 'linux' }
    steps { sh './smoke-test.sh https://staging.example.com' }
}

stage('Approve') {                                  // manual gate before production
    steps {
        timeout(time: 1, unit: 'HOURS') {
            input message: 'Deploy to production?', ok: 'Deploy'
        }
    }
}

stage('Deploy Prod') {
    when { branch 'main' } // CD only from main
    agent { label 'linux' }
    steps { sh "./deploy.sh prod ${GIT_COMMIT}" }
}

}

post {
    always { archiveArtifacts artifacts: 'reports/**', allowEmptyArchive: true }
    success { echo 'Released ✓' /* e.g. slackSend / emailxext here */ }
    failure {

```

```
    echo 'Pipeline failed – rolling back staging'
    sh './rollback.sh staging || true'
  }
}
```

How this pipeline embodies the principles

Build once: the image is built in the Build stage tagged with the commit, then that same tag is tested, pushed, and deployed — never rebuilt per environment.

Fast feedback & fail fast: Build and the parallel Test stage come first; if they fail, Quality Gate and everything after never run.

Parallelism: unit and integration suites run at the same time on separate agents, cutting total time.

Secrets: the registry password comes from Credentials via withCredentials and is masked in logs — never hard-coded.

Gated deployment: an input step requires a human to approve production, and when { branch 'main' } ensures only main deploys to prod.

Safety & reporting: post always archives reports, failure rolls back staging and (in a real setup) notifies the team, and buildDiscarder keeps disk usage sane.

Keep environment-specific details (URLs, credentials, replica counts) in `deploy.sh`, `Credentials`, or a config file — not hard-coded in the Jenkinsfile. The same pipeline then works for every service and environment.

Scenario: For the shop-api service, Aisha's team pushes to a feature branch: Jenkins builds the image once, runs unit + integration tests in parallel, checks coverage, and pushes the artifact — all green in four minutes. After merge to main, the same pipeline deploys to staging, runs smoke tests, pauses for a one-click approval, then deploys to production and tags the release. When a later smoke test fails, the failure block rolls staging back automatically and pings the team.

10. Pipeline Examples by Technology

The patterns are the same across languages — only the build/test commands change. Here are ready-to-adapt pipelines for the stacks a beginner meets most often. Copy one, swap in your repo's commands, and you have working CI.

Java / Maven

Maven projects build and test with `mvn`. The **tools** block lets Jenkins provide a specific JDK and Maven version (configured under Manage Jenkins → Tools).

```
pipeline {
  agent any
  tools { maven 'Maven-3.9'; jdk 'JDK-21' }
  stages {
    stage('Build') { steps { sh 'mvn -B clean compile' } }
    stage('Test') { steps { sh 'mvn -B test' } }
    post { always { junit 'target/surefire-reports/*.xml' } }
    stage('Package') { steps { sh 'mvn -B package -DskipTests' } }
    post { success { archiveArtifacts 'target/*.jar' } }
  }
}
```

Notes: `-B` runs Maven in batch (non-interactive) mode for cleaner logs; test reports live in `target/surefire-reports`; the built `.jar` is archived so you can download it from the build page. For Gradle, swap the commands for `./gradlew build test`.

Python

Create an isolated virtual environment, install dependencies, then lint and test. Publishing a JUnit-format report lets Jenkins show pass/fail trends.

```
pipeline {
  agent any
  stages {
    stage('Setup') {
      steps {
        sh 'python3 -m venv .venv'
        sh '. .venv/bin/activate && pip install -r requirements.txt'
      }
    }
    stage('Lint') {
      steps { sh '. .venv/bin/activate && flake8 src/' }
    }
    stage('Test') {
      steps { sh '. .venv/bin/activate && pytest --junitxml=reports/pytest.xml' }
    }
  }
  post { always { junit 'reports/pytest.xml' } }
}
```

Each `sh` step runs in a fresh shell, so the virtualenv is not remembered between steps -- that is why every step re-runs the activate command. Alternatively, run the whole stage inside a Python Docker agent (see below).

Node.js

Use `npm ci` (not `npm install`) in CI — it installs the exact locked versions and is faster and reproducible.

```
pipeline {
  agent any
  stages {
    stage('Install') { steps { sh 'npm ci' } }
    stage('Lint')     { steps { sh 'npm run lint' } }
    stage('Test')     { steps { sh 'npm test' }
                        post { always { junit 'reports/*.xml' } } }
    stage('Build')    { steps { sh 'npm run build' }
                        post { success { archiveArtifacts 'dist/**' } } }
  }
}
```

Run inside a Docker image (any language)

Instead of installing tools on the agent, run a stage inside a container that already has them. This keeps agents clean and builds reproducible — everyone uses the exact same toolchain.

```
pipeline {
  agent { docker { image 'python:3.12' } } // whole pipeline runs in this image
  stages {
    stage('Test') { steps { sh 'pip install -r requirements.txt && pytest' } }
  }
}

# or per-stage, mixing images:
stage('Build') { agent { docker { image 'node:20' } } steps { sh 'npm run build' } }
```

Build & push a Docker image

Containerize the app and push it to a registry, tagging with the commit so every image is traceable. The registry password comes from Credentials.

```
pipeline {
  agent { label 'docker' }
  environment { IMAGE = 'registry.example.com/myapp' }
  stages {
    stage('Build image') {
      steps { sh "docker build -t $IMAGE:${GIT_COMMIT} ." }
    }
    stage('Push image') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'registry-login',
                                         usernameVariable: 'U', passwordVariable: 'P')]) {
          sh 'echo "$P" | docker login $IMAGE -u "$U" --password-stdin'
          sh "docker push $IMAGE:${GIT_COMMIT}"
        }
      }
    }
  }
}
```

Monorepo — build only what changed

In a repo holding several services, you usually do not want to rebuild everything on every push. Detect which paths changed and build those services in **parallel**. A simple approach uses **changeset** conditions:

```
pipeline {
  agent any
  stages {
    stage('Services') {
      parallel {
        stage('api') {
          when { changeset 'services/api/**' }
          steps { dir('services/api') { sh 'make build test' } }
        }
        stage('web') {
          when { changeset 'services/web/**' }
          steps { dir('services/web') { sh 'npm ci && npm test' } }
        }
        stage('worker') {
          when { changeset 'services/worker/**' }
          steps { dir('services/worker') { sh 'make build test' } }
        }
      }
    }
  }
}
```

Each service stage only runs when files under its folder changed (**changeset**), and **dir(...)** runs the commands inside that service's subfolder. Unchanged services are skipped, so the build stays fast as the monorepo grows.

Deploying to Kubernetes / AWS

A deploy stage usually just runs the right CLI with credentials from the Credentials store. Example deploying a container image to Kubernetes, gated to the main branch:

```
stage('Deploy') {
  when { branch 'main' }
  steps {
    withCredentials([file(credentialsId: 'kubeconfig', variable: 'KUBECONFIG')]) {
      sh "kubectl set image deployment/myapp myapp=$IMAGE:${GIT_COMMIT}"
      sh 'kubectl rollout status deployment/myapp --timeout=120s'
    }
  }
}
```

For AWS, the pattern is identical — store an access key (or use an IAM role on the agent) and run the AWS CLI, e.g. **aws ecs update-service ...** or **aws s3 sync dist/ s3://my-bucket**. The **rollout status** line makes the stage fail if the deployment does not become healthy, so a bad release is caught automatically.

Keep deploy logic in a script (**deploy.sh**) that the pipeline calls, rather than long inline shell. The Jenkinsfile stays readable, and you can test the script independently.

Scenario: Aisha's team runs a monorepo with an API (Java), a web app (Node), and a worker (Python). On each push, only the changed services build — in parallel, each inside its own language's Docker image. On main, changed services build an image, push it tagged by commit, and roll out to Kubernetes, with the stage failing automatically if the rollout is unhealthy.

PART C — ADVANCED: SCALE, SECURITY & SPEED

These topics are not needed on day one, but they are what make Jenkins fast, secure, and ready for a real team.

11. Agents, Credentials & Advanced Pipelines

Distributed builds with agents

What: agents are extra machines the controller sends work to, so many builds run at once and on the right OS. **When:** one Jenkins server is no longer enough, or you need to build on Windows, Linux, and macOS.

You connect an agent (via SSH, an inbound agent, or a cloud/Docker plugin), give it **labels** (e.g. linux, docker), then target it in a pipeline:

```
pipeline {
  agent { label 'linux' }           // run on any agent labelled "linux"
  stages {
    stage('Build') { steps { sh 'make' } }
  }
}

# run a stage inside a Docker image:
agent { docker { image 'node:20' } }
```

Credentials — handle secrets safely

What: never paste passwords or tokens into a job. Store them under **Manage Jenkins** →

Credentials; Jenkins keeps them encrypted and gives builds a reference ID, so secrets never appear in plain text. Use them in a pipeline like this:

```
withCredentials([string(credentialsId: 'deploy-token', variable: 'TOKEN')]) {
  // Jenkins injects the secret into TOKEN for this block only
  // and masks it in the console log.
  sh './deploy.sh --token "$TOKEN"'
}
```

Secrets are masked in logs, but a careless echo "\$TOKEN" can still reveal them. Never print a secret, and never commit one to the Jenkinsfile.

Parallel stages

What: run independent work at the same time to finish faster — e.g. run tests on two platforms at once.

```
stage('Test') {
  parallel {
    stage('Linux') { agent { label 'linux' } steps { sh 'npm test' } }
    stage('Windows') { agent { label 'windows' } steps { bat 'npm test' } }
  }
}
```

Shared libraries

What: reusable pipeline code shared across many projects, kept in its own Git repo. **When:** several teams repeat the same pipeline steps and you want one place to maintain them. You load it at the top of a Jenkinsfile:

```
@Library('my-shared-library') _  
  
pipeline { /* now you can call steps defined in the library */ }
```

Scenario: HelloApp grows into three services. Aisha adds two Linux agents and a Windows agent, runs tests in parallel across them, pulls the deploy token from Credentials, and moves the common build steps into a shared library her whole team reuses.

12. Plugins, Security & Health

Managing plugins

Almost every capability comes from a plugin. Manage them under **Manage Jenkins → Plugins** (Available / Installed / Updates tabs). Useful ones: Git, Pipeline, Blue Ocean (a nicer pipeline UI), Docker, and a notifier such as Slack or Email.

Install only the plugins you need and keep them updated. Every extra plugin is more code to maintain and a potential security concern.

Security basics

- Turn on security and create real user accounts — never leave Jenkins open to the whole network.
- Give users only the access they need (role-based permissions).
- Keep Jenkins, Java, and plugins updated for security fixes.
- Run builds on agents, not the controller, so a misbehaving build cannot harm the main server.
- Store all secrets in Credentials; never in job config or the Jenkinsfile.

Backups & health

Everything Jenkins knows lives in its home directory (**JENKINS_HOME**) — jobs, config, build history, credentials. Back it up regularly; restoring is as simple as putting it back. Watch disk space (old builds and artifacts add up) and set each job to discard old builds after a limit.

Configuration as Code (JCasC)

Advanced teams define the entire Jenkins setup in a YAML file with the **Configuration as Code** plugin, so a Jenkins server can be rebuilt identically from version control — the same “everything as code” idea as the Jenkinsfile, applied to Jenkins itself.

Scenario: Before going to production, Aisha turns on security with real logins, limits who can configure jobs, sets jobs to keep only the last 20 builds, schedules a nightly backup of JENKINS_HOME, and captures the server setup in a Configuration-as-Code file.

13. Quick Reference & Cheat Sheet

A fast reference grouped by level. Keep this page open while you work.

Simple — getting started

Term / Command	What it does	When to use it
<code>http://localhost:8080</code>	The Jenkins web dashboard.	Open Jenkins.
<code>initialAdminPassword</code>	First-run unlock password file (in secrets/).	First launch.
New Item	Create a job or pipeline.	Start a new job.
Build Now	Run a job once, by hand.	Test a job.
Console Output	Line-by-line log of a build.	See why a build passed/failed.
Freestyle project	Form-based job, no code.	Your very first job.

Moderate — pipelines & triggers

Term / Command	What it does	When to use it
<code>Jenkinsfile</code>	Pipeline defined as code, in the repo root.	Any real project.
<code>pipeline { agent any }</code>	Declarative pipeline skeleton.	Standard pipeline.
<code>stage('X') { steps { } }</code>	One phase of the pipeline.	Build / Test / Deploy.
GitHub webhook	Build the instant you push.	Fast automatic builds.
<code>H/5 * * * * *</code>	Poll / schedule (cron) syntax.	Timed or polled builds.
<code>parameters { }</code>	Ask for input before building.	Choose env, version, etc.
<code>post { success/failure }</code>	Run steps based on result.	Notify, clean up.

Advanced — scale & secure

Term / Command	What it does	When to use it
<code>agent { label 'linux' }</code>	Run on a specific agent.	Distributed / multi-OS builds.
<code>agent { docker { } }</code>	Run inside a container image.	Clean, reproducible builds.
<code>parallel { }</code>	Run stages at the same time.	Speed up independent work.
<code>withCredentials { }</code>	Inject a stored secret safely.	Deploy keys, tokens.
<code>@Library('name')</code>	Load shared pipeline code.	Reuse across teams.
<code>JENKINS_HOME</code>	All config/jobs/history live here.	Back this up.
<code>when { branch 'main' }</code>	Run a stage conditionally.	Deploy only from main.

Minimal Jenkinsfile template

```

pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'npm install' } }
    stage('Test') { steps { sh 'npm test' } }
  }
  post { failure { echo 'Build failed' } }
}

```

14. Mistakes, Habits & Next Steps

Common mistakes

- Wrong Java version — Jenkins silently refuses to start. Check `java -version` matches what your release needs.
- Reading console output top-down — the real error is almost always near the bottom; start there.
- Hard-coding secrets in jobs instead of using the Credentials store (and accidentally echoing them in logs).
- Running everything on the controller — heavy builds can slow or crash the whole server.
- Giant do-everything jobs — break work into clear stages so failures are easy to locate.
- Forgetting a webhook needs Jenkins reachable from GitHub; on a laptop, polling may be simpler to start.

- Never cleaning up — old builds and artifacts fill the disk; set jobs to discard old builds.

Good habits

- Keep the pipeline as a Jenkinsfile in the repo, versioned with the code.
- Make builds fast and run them often — quick feedback is the whole point of CI.
- Treat a red build as “stop and fix”, not “ignore and continue”.
- Store secrets in Credentials; run builds on agents; keep Jenkins, Java, and plugins updated.
- Back up JENKINS_HOME regularly.
- Start small: one Freestyle job, then a simple pipeline, then triggers and agents. Grow as you understand each piece.

Where to go next

- Add build badges and notifications (Slack/email) so the team sees results fast.
- Explore Blue Ocean for a friendlier pipeline visualization.
- Learn Configuration as Code (JCasc) to version your Jenkins setup itself.
- Read the official docs at jenkins.io/doc for depth on any topic.

Scenario: A few weeks in, Aisha pushes code, a webhook fires, Jenkins runs the HelloApp pipeline across parallel agents, three stages go green, artifacts are archived, and the app deploys from main — all without her clicking anything. The Friday integration pain from Section 1 is gone for good.

You now know Jenkins from first install to agents, credentials, and pipelines as code. Keep this guide handy, practise often, and CI/CD will become second nature.